Lab 06 - Block Ciphers - Baby Feistel and DES

Intro

In lecture we discussed the differences between stream ciphers and block ciphers. We spent some time looking at older stream ciphers such as A5/1 which was used in GSM (gen 2 cell phones) and RC4 (which was used in WEP and SSL/TLS), as well as the newer ChaCha20-Poly1305 whose use has been widely included in hardware, software, and protocols. We also are looking at block ciphers such as DES and AES and the different modes block ciphers can run in.

The most well-known and influential Feistel cipher was the Data Encryption Standard (DES). Also, the Soviet Union/Russia had a similar cipher named GOST. While DES has been deprecated, there are still numerous Feistel ciphers available for use today. The better known are Blowfish, Twofish (one of the finalists to become AES), CAST (also a contender to be AES), and Camellia.

There is a Tiny Encryption (TEA) algorithm that is commonly used to simply demonstrate a Feistel algorithm. A Baby DES implementation also exists. Both of those have more specifications than we need to implement in lab. We just want to get an understanding of a Feistel network. We will implement a smaller, simple 4-bit Baby Feistel cipher and only run it for three rounds. This is also a commonly implemented Feistel network in labs and classrooms as a teaching device.

Additionally, you will conduct a meet in the middle attack on 2DES and you will use 2-key 3DES to decrypt a ciphertext.

Part 01

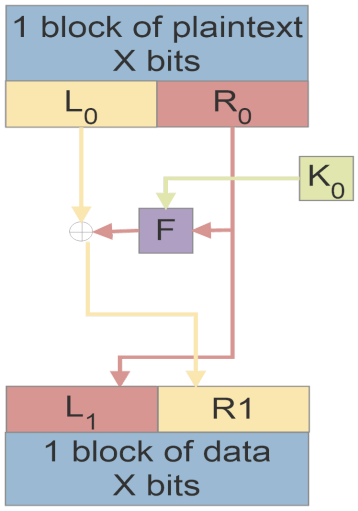
You will implement a 4-bit block cipher with a 4-bit key and output 4-bits of data. It will run for 3 rounds. You will not be implementing a block mode so processing a full message won't be possible. You will be working with the first 4-bits of a single character since you are working at the binary level. Remember, even if you implemented a block cipher mode and could process an entire message, this is a very simple Feistel cipher which, of course, is not secure.

To get the skeleton code for all three parts of the lab:

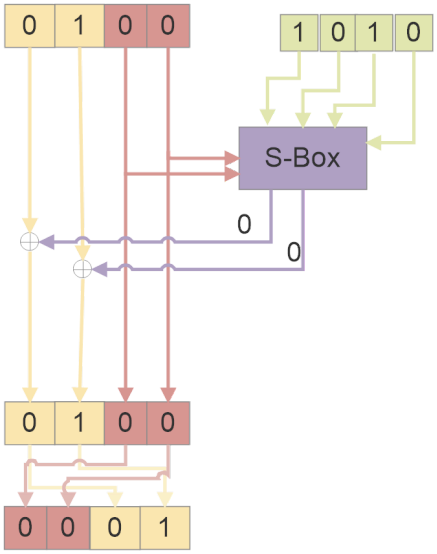
```
scp -r repo@repo.homework.3310.com:/home/repo/lab06 .
```

Remember you will have to give yourself permissions to edit the files.

1. Remember how a Feistel network functions? The images used in lecture are provided below for reference.
    a. The function starts by splitting the block of X bits (in this case 4 bits) into two parts, a left half (L_0) and a right half (R_0)
    b. After it is split into halves, the round function (F) is applied to one half using a round key. In DES and in our 4-bit implementation this will be the right half (R_0).
    c. The output of the round function is XOR'd with the other half. In our implementation this will be the left half (L_0).
    d. The halves are then transposed so that output of step c above becomes R_1 and the old R_0 moves down to become L_1.



2. **As an example, we will use 0100 as the plaintext and the key of 1010.** The R_0 bits are input to the S-Box and the L_0 bits are XOR'd with the output from the S-Box. The description of how the S-Box works is in the next step. The final step is to swap the halves, making the bits in the L_0 new right half and the bits in the R_0 position the new left half. These bits become L_1 and R_1 in the next round.



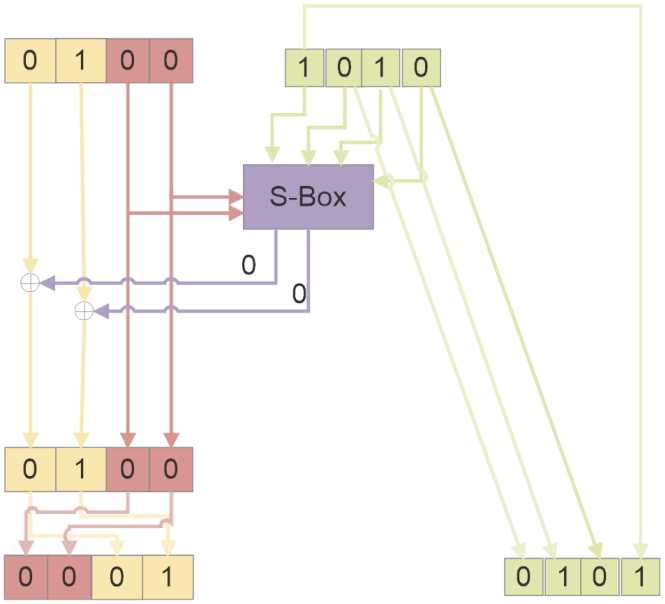
3. The S-Box we are using is a simple lookup table. It is comparable to what happens in DES. In our version, you use the key value as the index to the rows and the two bits from R_0 as the index to the columns. For this example, the row value is 1010 and the column value is 00. The intersection of those two provides you with 00 as the output.

		Last 2 bits of block		
key	00	01	10	11
0000	00	10	10	11
0001	10	11	00	00
0010	01	10	00	00
0011	11	00	01	11
0100	10	00	11	01
0101	01	11	10	10
0110	11	01	11	10
0111	00	01	10	01
1000	11	01	00	10
1001	01	00	01	11
1010	00	11	11	00
1011	01	10	11	01
1100	10	11	01	10
1101	11	01	00	00
1110	00	00	10	01
1111	10	10	01	11

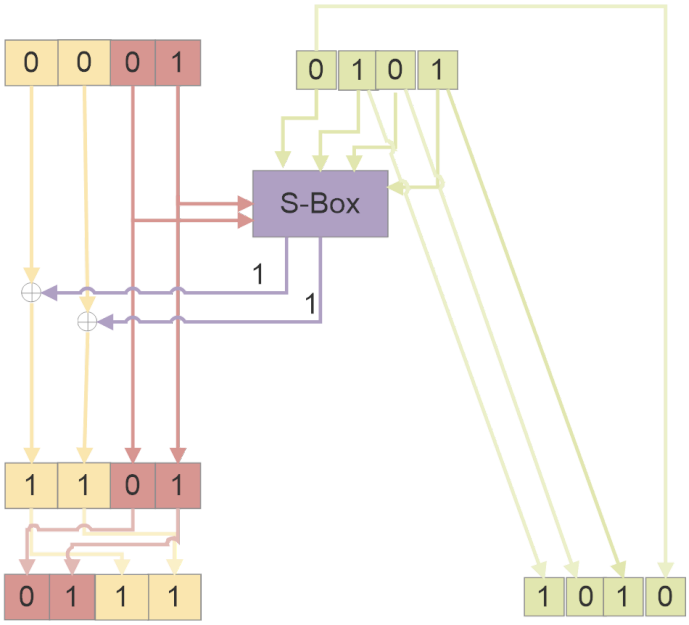
In the skeleton code I give you a list where the key is an integer index to the row and the output is an integer value. **Do not modify the sBox list.**

```
sBox = [[0]*4]*16
sBox[0] = [0, 2, 2, 3]
sBox[1] = [2, 3, 0, 0]
sBox[2] = [1, 2, 0, 0]
sBox[3] = [3, 0, 1, 3]
sBox[4] = [2, 0, 3, 1]
sBox[5] = [1, 3, 2, 2]
sBox[6] = [3, 1, 3, 2]
sBox[7] = [0, 1, 2, 1]
sBox[8] = [3, 1, 0, 2]
sBox[9] = [1, 0, 1, 3]
sBox[10] = [0, 3, 3, 0]
sBox[11] = [1, 2, 3, 1]
sBox[12] = [2, 3, 1, 2]
sBox[13] = [3, 1, 0, 0]
sBox[14] = [0, 0, 2, 1]
sBox[15] = [2, 2, 1, 3]
```

4. Before you enter the next round, you need to do a key rotation. In DES this is called the round key. Again, we will do a simple process of moving the left most bit in the key to the right most position while shifting the remaining 3 bits one position to the left.

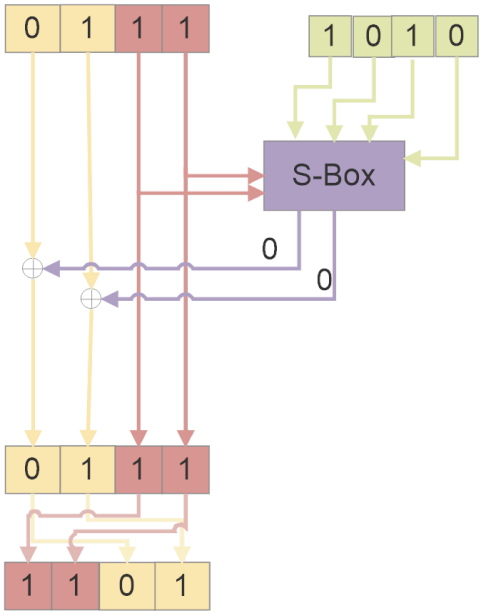


5. You now start the second round with 0001 as your block of input and 0101 as your round key. The image illustrates the process.



	Last 2 bits of block			
key	00	01	10	11
0000	00	10	10	11
0001	10	11	00	00
0010	01	10	00	00
0011	11	00	01	11
0100	10	00	11	01
0101	01	11	10	10
0110	11	01	11	10
0111	00	01	10	01
1000	11	01	00	10
1001	01	00	01	11
1010	00	11	11	00
1011	01	10	11	01
1100	10	11	01	10
1101	11	01	00	00
1110	00	00	10	01
1111	10	10	01	11

6. The third round is shown in the image below. Notice we don't need to generate a next round key since we are only doing 3 rounds.



key	Last 2 bits of block			
	00	01	10	11
0000	00	10	10	11
0001	10	11	00	00
0010	01	10	00	00
0011	11	00	01	11
0100	10	00	11	01
0101	01	11	10	10
0110	11	01	11	10
0111	00	01	10	01
1000	11	01	00	10
1001	01	00	01	11
1010	00	11	11	00
1011	01	10	11	01
1100	10	11	01	10
1101	11	01	00	00
1110	00	00	10	01
1111	10	10	01	11

7. The 4-bits of ciphertext for this simple demonstration is 1101 which is the output from the final round. You should print your output for each round similar to what is below for your own troubleshooting and for the lab report.

   Round 1: [0, 0, 0, 1]
   Round 2: [0, 1, 1, 1]
   Round 3: [1, 1, 0, 1]

```
4-bit input to encode: 0100
4-bit key: 1010
Encode:
Round   0   0001
Round   1   0111
Round   2   1101
```

8. **The 4-bits of input you will use is 0001 and the key is 1100.** You need to write a program in python using `part01_skel.py` that will **encrypt** the 4-bit message. Do not hard code this. There are TODO: sections in each of the following functions
    a. `encodeRoundFunction`
    b. `encodeRotateKey`
    c. `encode`

9. You run the program with the command `python3 part01_skel.py`.

10. **<u>You will upload your commented python code to Canvas.</u>**

11. **<u>You will include your 4-bits of ciphertext in your lab report.</u>**

12. **<u>Also, include a screenshot of your code successfully encrypting the 4-bit message in the lab report.</u>**

Part 02

In this lab, we will explore the concept of the meet-in-the-middle (MITM) attack on 2DES (Double DES) encryption. 2DES is a variant of the Data Encryption Standard (DES) that applies the DES encryption algorithm twice with two different keys, each 56 bits in length, resulting in a total key size of 112 bits. Although this increases the key space beyond the 56-bit key used in standard DES, it does not offer the expected level of security due to the vulnerability to meet-in-the-middle attacks.

A meet-in-the-middle attack is a cryptographic technique that significantly reduces the complexity of breaking 2DES by taking advantage of the double encryption process. Instead of attempting to brute-force the full 112-bit key space, the meet-in-the-middle attack allows us to break the encryption in 2^56 operations, which is equivalent to breaking single DES.

In 2DES, a plaintext is encrypted with key1 and then the output is encrypted again with key2. The attack exploits the fact that the encryption consists of two stages. The attacker can intercept a known plaintext and its corresponding ciphertext, and then:

- Encrypt the known plaintext with all possible keys for the first DES stage (`key1`), storing the intermediate results.

- Decrypt the ciphertext with all possible keys for the second DES stage (`key2`), looking for a match with the intermediate values.

By finding a match between the results of encryption and decryption, the attacker can deduce both key1 and key2.

You will be working with a smaller key space to make the meet-in-the-middle (MITM) attack on 2DES computationally feasible within the lab environment. By limiting the key space to range(4) for each byte, we significantly reduce the number of possible keys. This results in 65536 possible keys (4^8), a far more manageable key space compared to the full 2^56 DES key space.

To put this into perspective, a full 2^56 key search using DES would involve approximately 72 quadrillion possible keys. With modern hardware capable of performing 10 million DES operations per second, searching the full DES key space would take about 228.6 years to complete. Even with faster hardware capable of 100 million DES operations per second, it would still take around 22.9 years to break the full key space.

For 2DES the brute force is about 2^112 which is astronomically infeasible. However, MITM reduces time complexity to 2^57. With 10 million DES operations per second, searching the full 2DES /MITM key space would take about 456 years to complete. With 100 million DES operations per second, it would still take around 45.6 years to break the full key space for 2DES/MITM.

With this reduced key space of 65536 keys you will be using, the program should execute much faster, allowing students to observe the MITM attack in action within just a few seconds to minutes on most modern machines, depending on hardware speed.

13. You need to install pycryptodome to run this lab. Since 2DES is not a secure algorithm it is not included in the cryptography package you previously installed.

    ```
    pip install pycryptodome
    ```

    OR

    ```
    sudo apt install python3-pycryptodome
    ```

14. You will be modifying three functions in the code for part02. There are TODO: sections in each of the following functions. You might also have to change `from Cryptodome.Cipher import DES` to `from Crypto.Cipher import DES` depending on the OS you are using (it will give you an error if it's wrong). You will use the pycryptodome documentation for single DES and electronic codebook block cipher mode.
    https://www.pycryptodome.org/src/cipher/des#module-Crypto.Cipher.DES
    a. `desEncrypt`
    b. `desDecrypt`
    c. `doubleDesEncrypt`
    d. `doubleDesDecrypt`

15. After modifying those functions, the rest of the provided code will recover the two original keys. It will also check that your `doubleDesEncrypt` and `doubleDesDecrypt` functions were implemented properly. The syntax to run the program is
`python3 part02_mitm_2DES_skel.py part02_ciphertext.txt`

16. **Include a screenshot of the two keys and the matching ciphertexts and plaintexts in your lab report.**

17. **What is a known flaw with the block cipher mode being used?** Please answer in your lab report.

18. **Why is known plaintext important in a meet-in-the-middle attack? What role does it play in recovering the encryption keys?** Please answer in your lab report.

19. **Could this attack be extended to 2-key 3DES? What about 3-key 3DES? What differences or additional challenges would arise in attacking 3DES with a similar approach?** Please answer in your lab report.

20. **Upload your code.**

Part 03

As discussed in lecture Triple DES (3DES) is a legacy cipher that applies the encryption algorithm three times to each block of text. It was created to provide a larger key space when DES was weighed, measured, and found wanting.

You will use 2-key Triple DES (3DES), an enhancement of the original Data Encryption Standard (DES). While 3-key 3DES uses three distinct keys, 2-key 3DES employs only two keys, with the first key reused for both the first and third stages of encryption. The encryption process in 2-key 3DES follows an Encrypt-Decrypt-Encrypt (EDE) sequence: data is first encrypted with key 1, then decrypted with key 2, and finally re-encrypted with key 1. This approach increases security compared to single DES by effectively doubling the key size,

making brute-force attacks more difficult. 2-key Triple DES was more widely used than 3-key.

In this exercise, you will be provided with a ciphertext that has been encrypted using 2-key 3DES, along with the corresponding key and initialization vector (IV). Your task is to decrypt the ciphertext using the given key and IV to recover the original plaintext.

You will only be doing this in the lab to understand how it works. You would not use 3DES in the real world since the key space is so small.

21. You need to use the pycryptodome documentation again to use 3DES to create the cipher object in CBC mode. https://pycryptodome.readthedocs.io/en/latest/src/cipher/des3.html#triple-des

22. The following functions need to be modified
    a. `encrypt`
    b. `decrypt`

23. Simply use the syntax `python3 part03_2_key_3DES_skel.py <mode> part03_ciphertext.bin part03_key.bin part03_iv.bin part03_decrypted_plaintext.txt`
    a. <mode> is encrypt or decrypt
    b. You will only decrypt in this lab

24. **Include a screenshot of the plaintext result in your lab report.** You may modify the program to print it to the screen and take a screenshot or you can just open the output file `part03_plaintext.txt` and screenshot or copy that text.

25. **What is the flaw with the block cipher mode you are using in this implementation of Triple DES?** Please include your answer in your lab report.

26. **In 2-key 3DES, what is the *effective key length* for encryption? How does it compare to the effective key length of 3-key 3DES? What are those other bits used for?** Please include your answer in your lab report.

27. **Historically, has there been a way to use a single key in Triple DES where you set key1=key2=key3? Why would you want to do that? What is the effective key size when key1=key2=key3?** Please include your answer in your lab report.

28. **Upload your code.**

Lab 06 Template

Part 01

1. **Upload python code for babyFeistel cipher as `<netid>_part01.py`**
   (5 points)

2. **Final 4-bit ciphertext.**
   (5 points)

3. **Screenshot of encrypting.**
   (5 points)

Part 02

4. **Include a screenshot of the two keys and the matching ciphertexts and plaintexts in your lab report.**
   (5 points)

5. **What is a known flaw with the block cipher mode being used?**
   (5 points)

6. **Why is known plaintext important in a meet-in-the-middle attack? What role does it play in recovering the encryption keys?**
   (10 points total, 5 points each question)

7. **Could this attack be extended to 2-key 3DES? What about 3-key 3DES? What differences or additional challenges would arise in attacking 3DES with a similar approach?**
   (10 points total, 3.33 points each question)

8. **Upload your code as `<netid>_part02_mitm_2DES.py`**
   (10 points total)

Part03

9. **Include a screenshot of the plaintext result in your lab report.**
   (5 points)

10. **What is the flaw with the block cipher mode you are using in this implementation of Triple DES?**
    (5 points)

11. **In 2-key 3DES, what is the *effective key length* for encryption? How does it compare to the effective key length of 3-key 3DES? What are those other bits used for?**
    (15 points total, 5 points each question)

12. **Historically, has there been a way to use a single key in Triple DES where you set key1=key2=key3? Why would you want to do that? What is the effective key size when key1=key2=key3?**
    (15 points total, 5 points each question)

13. **Upload your code `<netid>_part03_2_key_3DES.py`**
    (5 points)